

UNIVERSIDADE DO ESTADO DE SANTA CATARINA

RELATÓRIO: IMPLEMENTAÇÃO DE GRAFO COM CARGA PRIMÁRIA - TEG0002

1. Equipe: Lucas dos Santos Menezes - main.c, relatório (1,2,3,4);
Vinicius Persike de Souza - leituraDeDados.c, main.c, relatório (5);
Vinícius Zapelini Mangrich - func.c, main.c, relatório (3);

2. Identificação da Tarefa:

A tarefa define a implementação de uma lista de adjacências, essa lista é um vetor que para cada posição possui uma lista encadeada com os índices a qual possui adjacências, a implementação é interessante para grafos carregados já que ocupa espaço $O(n+m)$, sendo n e m a quantidade de vértices e arestas, respectivamente;

Outro requisito é fornecer o maior e menor grau dos elementos do grafo, na lista de adjacências os graus de um elemento são obtidos pela quantidade de índices da lista encadeada referente a este elemento, já que cada índice dessa lista representa uma aresta partindo do elemento do vetor e chegando naquele determinado índice, lembrando que se o elemento i possuir em sua lista encadeada o índice j , o índice j possuirá na sua lista i também, ou seja, não deixaremos de contar os graus de algum elemento.

Para verificarmos se o grafo é multigrafo ou não, precisamos nos atentar a duas ocorrências que satisfazem a isso, a primeira seria um laço (um elemento possuir o próprio índice na sua lista de adjacências), ou então a lista de adjacências de algum elemento possuir dois índices repetidos.

Para verificar a conexidade precisamos analisar se existe um caminho entre quaisquer " n " de vértices, para isso, no código executamos a DFS (Depth-First Search) para ver se aquele elemento passou por outros, e também por quantos, e guardamos essa informação no vetor de visitados " vis ". A DFS é um algoritmo que avança o máximo possível pelos ramos do grafo até que não consiga mais, e então retorna para um caminho não acessado, este algoritmo irá verificar que alguns elementos não foram acessados na primeira vez em que ele rodou até o final, e então iniciará novamente no próximo menor

elemento que ainda não foi acessado anteriormente, até que todos os nodos tenham sido marcados pelo vetor de visitados.

3. Explicação conceitual:

A única entrada que o programa recebe é o arquivo csv, que é carregado pelo arquivo “leituraDeDados.c”, quando executar o programa, este arquivo fará a leitura dos dados e já alimentará o grafo de forma a deixá-lo pronto, depois os dados irão para a “main.c”, que mostrará de forma ordenada todos os requisitos esperados pelo trabalho, utilizando funções que foram implementadas no “func.c”. A lógica desses arquivos e funções serão explicadas de forma mais detalhada abaixo:

- **leituraDeDados.c** contém a lógica para a leitura do arquivo csv, foi feito com apenas uma função que faz a leitura do arquivo e retorna uma lista de adjacência, já construindo o grafo. É importante mencionar que o arquivo enviado no moodle está no formato csv, entretanto está com os nodos, separados por espaços, fugindo do padrão convencional, separados por vírgulas, portanto, tivemos que adaptar a função de leitura; para ler corretamente, as informações deveriam estar entre vírgulas, para que os dados fiquem em diferentes colunas. Para o teste ser verificado corretamente ou deve-se alterar no código para ler com espaço ao invés de vírgula, ou ainda, mudar o csv e colocar cada nodo em uma coluna diferente.
- O **func.c** contém todas as outras funções, que seria a struct lista, que permite utilizarmos a lista encadeada, e funções como dfs, e outros que verificam se é multigrafo, os graus, e também os componentes conexos.

No “func.c” temos as seguintes funções: “dfs”, “graus”, “multigrafo”:

- A **dfs** é utilizada para saber os componentes conexos do grafo, então é criado um vetor “**vis**” responsável por armazenar em cada nodo, qual família ele pertence, ou melhor, qual componente ele pertence, então passamos por todos os nodos verificando se eles já foram visitados, caso um nodo não tenha sido visitado chamamos a dfs, a mesma percorre e colore todos os nodos do componente conexo utilizando um número identificador diferente do componente anterior, armazenando em qual componente eles pertencem dentro do “**vis**”, ao final, basta verificar quantos

identificadores diferentes temos em todos os nodos, e também conseguimos contar quantos elementos têm em cada componente com a informação do identificador.

- A função **graus**, verifica o menor e o maior grau do grafo, basicamente passamos por todos os nodos e verificamos o tamanho da lista de adjacência, ou seja, contamos para todos os nodos qual é o menor e o maior tamanho de lista de adjacência, que corresponde o graus de cada vértice.
- Na função **multigrafo**, é verificado se o grafo é multigrafo e é contado quantos laços e arestas múltiplas têm, isso é feito passando por cada adjacência de cada nodo, para cada nodo é colocado as suas adjacências em um vetor dinâmico, após isso esse vetor é ordenado e utilizamos uma lógica de que se o elemento atual for igual ao elemento anterior, com certeza é uma aresta múltipla, então aumentamos o contador; para laços é somente verificar se existe adjacência igual ao nodo, dessa maneira, para colocarmos as adjacências no vetor e ordenar teríamos uma complexidade de $O(n * \log n)$ para cada nodo sendo “n” o tamanho da lista de adjacência do nodo atual. Foi o melhor jeito que encontramos de fazer, a primeira ideia foi fazer um vetor de frequência para verificar se todos elementos repetidos na adjacência, ou seja, arestas múltiplas, entretanto para fazer isso teríamos que colocar um limite para nosso vetor de frequência e criar com esse tamanho todo nó, o que no pior caso ocuparia memória igual uma matriz de adjacência, outro jeito seria passar por todas as adjacências e após isso ver o maior número e criar o vetor com o limite do maior número, nos casos médios isso poderia ajudar entretanto no pior caso que seria todos os nodos têm o maior nodo como adjacência, sempre criaríamos um vetor de frequência de tamanho do maior nodo, ou seja ocuparia memória igual uma matriz de adjacência novamente. Portanto, concluímos que o jeito que foi feito, se encaixaria melhor, ocupando menos espaço pelo fato de criar um vetor com tamanho no máximo quantidade de adjacências para cada nodo, por mais que adicione um “log” na complexidade de tempo, pelo fato de ter que ordenar o vetor.

Os demais arquivos são apenas *headers* e dados de entrada para que o programa funcione adequadamente.

4. Instruções sobre a compilação:

As IDE's utilizadas foram o VIM, Clion e o VSCode, referente às bibliotecas, utilizamos apenas as padrões do C (stdlib.h, stdio.h) e as *headers* implementadas que estão nos arquivos ".h".

Para compilar, é necessário baixar os seguintes arquivos: dados.csv, func.c, func.h, leituraDeDados.c, leituraDeDados.h e main.c. Dentro do editor use: "gcc main.c leituraDeDados.c func.c -o programa", e para executar: "./programa". Caso queira ver a lista de adjacências, no começo da main o primeiro "for" comentado é responsável por printar a lista.

Os arquivos ".c" são o código e a lógica para resolução do problema, os arquivos ".h" são as *headers*, que carregam a declaração das funções utilizadas nos arquivos ".c", e o arquivo dados.csv é onde colocamos informações para serem carregadas no grafo, é importante ressaltar a necessidade de colocar os dados no formato "v1 , v2" onde v(i) seriam os índices correspondentes aos vértices ligados pela aresta formada.

5. Descrição/análise de resultados:

Através desse trabalho podemos concluir que, o desenvolvimento do código é essencial para identificação da estrutura do grafo e de suas propriedades, já que graficamente, como foi representado pelas imagens 2 e 3, fica complicado de entender apenas visualmente, sendo um papel crucial a implementação de funções para o entendimento acerca da teoria de grafos. A utilização da DFS permitiu identificar corretamente os componentes conexos do grafo, demonstrando na prática o funcionamento do algoritmo estudado em sala. A estratégia de marcação dos vértices visitados garantiu que todos os vértices pertencentes ao mesmo componente fossem agrupados corretamente.

```
PS C:\Users\Usuario\Desktop\UDESC\TEG> ./teste
Abriu o arquivo.

=====

Maior grau: 3 -- Menor grau: 1

=====

Nao eh multigrafo!

=====

0 grafo tem 3 componentes conexos
0 tamanho do componente 1 eh: 250
0 tamanho do componente 2 eh: 499
0 tamanho do componente 3 eh: 250

=====
```

Imagem 1: Captura de tela do terminal com os resultados obtidos pelo código

Grafo 2D

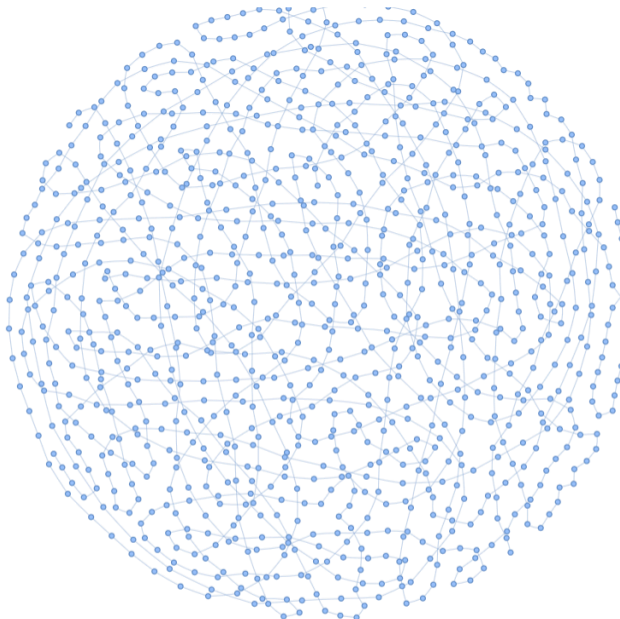


Imagem 2: Repositório no Google Colab do script em python que estava no Moodle.

Grafo 3D

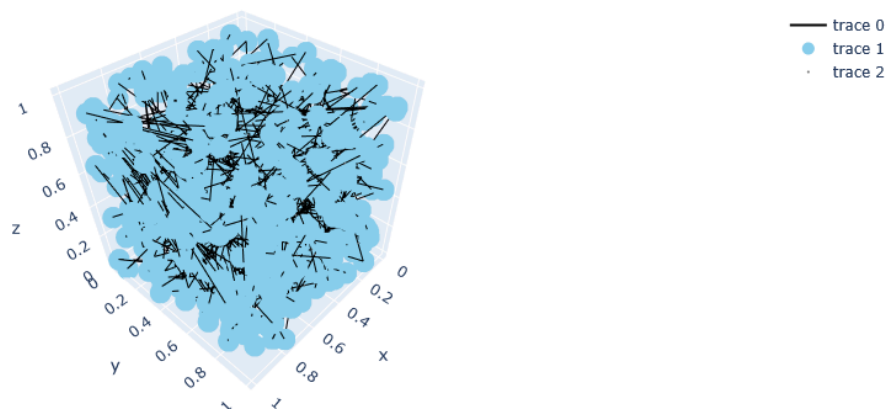


Imagem 3: Repositório no Google Colab do script em python que estava no Moodle.

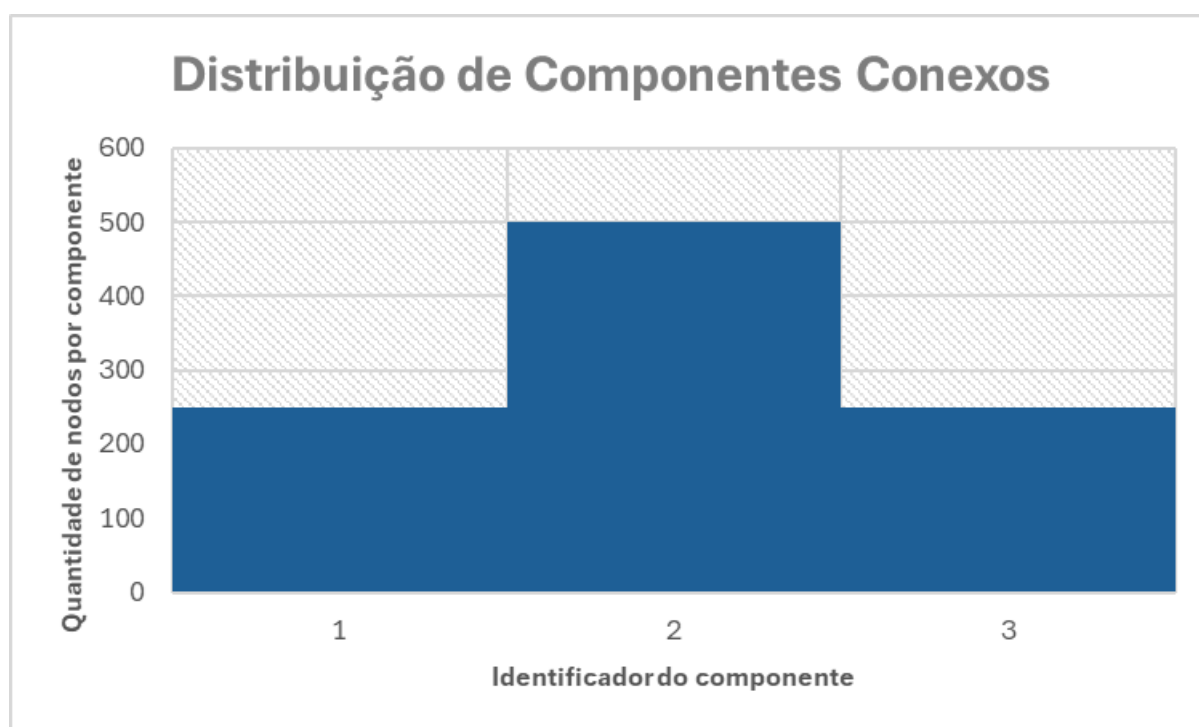


Imagem 4: Gerado no Excel através dos dados do código, feito por Vinícius Persike.